
Projet L1 Informatique

Introduction à la Programmation Orientée Objet

Dictionnaire intelligent et correcteur orthographique

basé sur l'utilisation d'un arbre lexicographique

Formation : L1 CMI Informatique

Établissement : CERI Avignon Université (UAPV)

Dépôts prévus : 15 avril, 30 avril, 15 mai, soutenance

Semestre 2 Année universitaire 2025-2026

Table des matières

1	Présentation générale	2
1.1	Structure du projet	2
1.2	Rendu	2
2	Partie 1 Arbre lexicographique (arbre en partie commune)	3
2.1	Définition formelle	3
2.2	Exemple	3
2.3	Structures de données	4
2.4	Marche à suivre	4
3	Partie 2 Tableau dynamique	5
3.1	Structures	5
3.2	Marche à suivre	5
4	Partie 3 Comparaison des deux structures	5
5	Fonctionnalités obligatoires (110)	5
6	Fonctionnalités avancées (1116)	6
6.1	Fonctionnalité 11 Autocomplétion par préfixe	6
6.2	Fonctionnalité 12 Saisie de texte assistée	6
6.3	Fonctionnalité 13 Distance de Levenshtein	6
6.3.1	Distance de similarité	6
6.3.2	Algorithme (programmation dynamique)	7
6.4	Fonctionnalité 14 Correction orthographique via l'arbre	8
6.4.1	Principe	8
6.4.2	Optimisation clé : élagage (<i>pruning</i>)	8
6.4.3	À implémenter	8
6.5	Fonctionnalité 15 Correction d'un fichier texte	8
6.6	Fonctionnalité 16 Interface interactive en terminal	9
7	Récapitulatif des fonctionnalités	9

1 Présentation générale

L'objet de ce projet est d'étudier deux façons de stocker un lexique en utilisant deux structures de données différentes : un **arbre lexicographique** et un **tableau dynamique**. Les deux méthodes seront comparées du point de vue de l'encombrement mémoire et de la vitesse d'accès.

On y ajoutera ensuite des fonctionnalités de correction orthographique reposant sur la **distance de Levenshtein**.

1.1 Structure du projet

Le projet se découpe en **au moins 6 fichiers** :

- `apc.cpp` et `apc.h` données et fonctions de gestion de l'arbre lexicographique ;
- `tab.cpp` et `tab.h` gestion du tableau dynamique ;
- `principal.cpp` programme principal lançant la création du dictionnaire dans l'arbre et dans le tableau ;
- `makefile` compilation de l'ensemble.

1.2 Rendu

Attention

Votre travail doit pouvoir être **compilé** sur les ordinateurs du CERI avec la commande `make`, et **exécuté** sur les machines du CERI.

Quatre dépôts sont prévus : le **15 avril**, le **30 avril**, le **15 mai**, puis un **dépôt final** le jour de la soutenance (à la dernière séance de l'AMS). Chaque dépôt doit être accompagné d'un README expliquant la progression depuis le précédent dépôt. Le correcteur veillera à ce que ces rendus témoignent d'une évolution continue. **Cette progression constitue un critère essentiel dans l'évaluation.**

Pour préparer le rendu, dans un terminal :

Terminal

```
$ mkdir Nom-Prenom
$ cp *.cpp *.h makefile Nom-Prenom/
$ tar cvfz Nom-Prenom.tgz Nom-Prenom/
```

Attention

Seuls les fichiers .tgz peuvent être déposés. Remplacer `<Nom>` par votre nom et `<Prenom>` par votre prénom.

2 Partie 1 Arbre lexicographique (arbre en partie commune)

Un **arbre lexicographique** permet de stocker, dans une structure d'arbre, un ensemble de mots en factorisant au maximum les débuts de mots, afin d'utiliser le moins d'espace mémoire possible tout en permettant un accès efficace.

2.1 Définition formelle

- Les **nuds** portent des caractères comme information.
- La **racine** est conventionnelle et ne porte aucune information significative.
- Les **fil de la racine** portent les premières lettres des mots, rangées par ordre alphabétique.
- Plus généralement, soit $r_0 r_1 r_2 \dots r_n$ un chemin depuis la racine (r_0 est la racine), $c_1 c_2 \dots c_n$ les lettres rangées dans ces nuds. Les fils de r_n sont des nuds portant les $(n+1)^{\text{èmes}}$ lettres des mots commençant par $c_1 c_2 \dots c_n$, rangées par ordre alphabétique.
- Tous les mots sont terminés par le caractère spécial ' $\backslash 0$ '.

2.2 Exemple

L'arbre ci-dessous correspond aux mots : *main*, *mais*, *mal*, *male*, *mon*, *son*, *sono*, *sons*, *sont*.

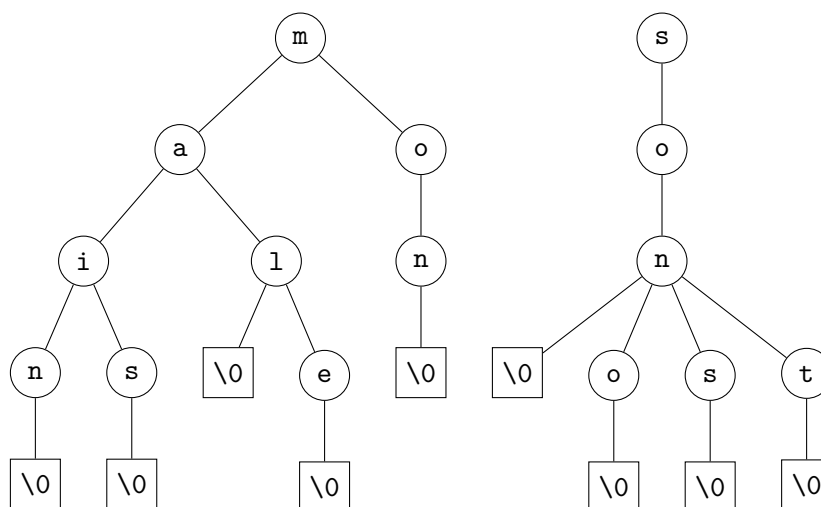


FIGURE 1 – Arbre lexicographique pour {*main*, *mais*, *mal*, *male*, *mon*, *son*, *sono*, *sons*, *sont*}

2.3 Structures de données

Les arbres lexicographiques ont un nombre de fils quelconque par nud. On utilise la représentation **filstrère** :

```
1 class noeud;  
2  
3 class arbre {  
4     noeud* racine;  
5 public:  
6     // ...  
7 };  
8  
9 class noeud {  
10     friend class arbre;  
11     char    info;  
12     noeud* fils;    // descendre dans l'arbre  
13     noeud* frere;   // parcourir un niveau  
14 public:  
15     // ...  
16 };
```

Listing 1 – Structures noeud et arbre

Chaque nud possède deux pointeurs : un pour **descendre** dans l'arbre (fils), un pour **parcourir** un niveau (frère).

2.4 Marche à suivre

Lire tous les mots du dictionnaire un par un et les charger dans l'arbre au fur et à mesure.

3 Partie 2 Tableau dynamique

Le lexique est rangé dans un **tableau dynamique ordonné** afin de permettre une recherche par dichotomie.

3.1 Structures

```
1 char** Tab;
2 char*  buf;
```

Listing 2 – Tableau dynamique de chaînes

Chaque case de `Tab` est un `char*` pointant vers un mot stocké dans le grand tableau continu `buf`.

Tab[0]	Tab[1]	Tab[2]	...
↓	↓	↓	
buf : a b c \0	a b m \0	a c p \0	...

3.2 Marche à suivre

1. Déterminer la taille nécessaire pour stocker tous les mots.
2. Allouer `buf`.
3. Charger la totalité des mots dans `buf`.
4. Déterminer le nombre de mots du dictionnaire.
5. Allouer `Tab`.
6. Faire `í` pointer `z` les pointeurs de `Tab` sur les mots de `buf`.
7. Trier `Tab` en ne déplaçant que des pointeurs.

4 Partie 3 Comparaison des deux structures

1. Estimer le **temps de préparation** des deux structures (chargement du dictionnaire).
2. Estimer le **temps de recherche** de tous les mots du fichier `test.dic` dans chacune des deux structures.

Remarque

Le fichier `test.dic` peut contenir des mots absents de `lex.dic`.

5 Fonctionnalités obligatoires (110)

Les fonctionnalités suivantes doivent être implémentées dans la classe `Arbre` :

1. Constructeur sans argument
2. Constructeur avec un nom de fichier comme argument
3. Destructeur
4. Constructeur par recopie
5. Insertion d'un mot dans l'arbre lexicographique
6. Recherche exacte d'un mot dans l'arbre lexicographique
7. Suppression d'un mot de l'arbre lexicographique
8. Comptage du nombre total de mots dans l'arbre lexicographique
9. Calcul de la longueur du mot le plus long
10. Sauvegarde du dictionnaire contenu dans l'arbre dans un fichier

6 Fonctionnalités avancées (1116)

6.1 Fonctionnalité 11 Autocomplétion par préfixe

Écrire une **fonction membre** de la classe `Arbre` qui prend en argument un début de mot (préfixe) et qui renvoie tous les mots commençant par ce préfixe.

Remarque

On suppose que le constructeur prenant un `nom_fichier_dictionnaire` en argument a déjà été appelé, c'est-à-dire que le dictionnaire est déjà représenté par l'arbre lexicographique.

6.2 Fonctionnalité 12 Saisie de texte assistée

Développer une fonctionnalité permettant à l'utilisateur de saisir un texte avec une **aide à la correction orthographique**.

- L'utilisateur saisit les premières lettres d'un mot, suivies d'une tabulation (ou d'un autre caractère déclencheur).
- Le programme propose alors k mots commençant par ce préfixe.
- Si l'utilisateur ne trouve pas le mot recherché, il peut appuyer de nouveau sur la tabulation pour afficher les suggestions suivantes.
- S'il trouve le mot, il peut le sélectionner puis continuer la saisie.
- Si le mot n'existe pas dans le dictionnaire, l'utilisateur peut le saisir entièrement ; ce mot sera alors ajouté au dictionnaire.
- Après chaque interaction, le texte complet doit être réaffiché.

Remarque

Le texte du fichier peut être rangé dans une **liste chaînée** (par exemple).

6.3 Fonctionnalité 13 Distance de Levenshtein

6.3.1 Distance de similarité

Nous souhaitons établir une distance entre deux mots. Trois types d'erreurs sont possibles, chacune augmentant la distance de **1 point** :

Opération	Exemple	Coût
Suppression	<code>distance("chien", "chen")</code>	1 pt
Insertion	<code>distance("chen", "chien")</code>	1 pt
Substitution	<code>distance("chuen", "chien")</code>	1 pt

6.3.2 Algorithme (programmation dynamique)

Soient deux mots w et v de tailles respectivement n et m . On construit une matrice D de taille $(m+1) \times (n+1)$.

Initialisation :

$$D[0][j] = j \quad (\text{insérer } j \text{ caractères}) \quad D[i][0] = i \quad (\text{supprimer } i \text{ caractères})$$

Récurrence :

$$D[i][j] = \min \begin{cases} D[i-1][j] + 1 & (\text{suppression}) \\ D[i][j-1] + 1 & (\text{insertion}) \\ D[i-1][j-1] + \delta & (\text{substitution}) \end{cases}$$

avec

$$\delta = \begin{cases} 0 & \text{si } w_i = v_j \\ 1 & \text{sinon} \end{cases}$$

$D[i][j]$ représente la distance entre les i premières lettres de w et les j premières lettres de v .

Exemple : $v = \text{"chien"}$, $w = \text{"shein"}$. On initialise $D[0][j] = j$ et $D[i][0] = i$. La distance finale est $\text{distance}(w, v) = D[5][5] = 2$.

		s	h	e	i	n
	0	1	2	3	4	5
c	1	1	2	3	4	5
h	2	2	1	2	3	4
i	3	3	2	2	2	3
e	4	4	3	2	3	3
n	5	5	4	3	3	2

TABLE 1 – Matrice D pour $v = \text{"chien"}$ et $w = \text{"shein"}$ résultat : $D[5][5] = 2$

Exercice : étant donnés deux mots, calculer la distance entre eux en utilisant l'algorithme ci-dessus. Tester la fonction dans différents cas d'utilisation.

6.4 Fonctionnalité 14 Correction orthographique via l'arbre

6.4.1 Principe

Pour corriger un mot hors vocabulaire, on souhaite trouver les k mots du dictionnaire les plus proches selon la distance de Levenshtein.

Stratégie sur l'arbre : à chaque nud de l'arbre lexicographique, on maintient **une ligne de la matrice D** :

$$D[j] = \text{distance}([w_0, w_1, \dots, w_j], \text{le_prefixe_courant})$$

Initialisation (à la racine, préfixe vide) :

$$D[j] = j \quad \text{pour } j \text{ allant de } 1 \text{ à } m.$$

Descente dans l'arbre (le caractère courant du nud est c) :

$$D'[0] = D[0] + 1$$

$$D'[j] = \min \begin{cases} D'[j-1] + 1 & (\text{insertion}) \\ D[j] + 1 & (\text{suppression}) \\ D[j-1] + \delta & (\text{substitution}) \end{cases}$$

avec

$$\delta = \begin{cases} 0 & \text{si } w_j = c \\ 1 & \text{sinon} \end{cases}$$

Que faire quand on trouve un mot ? À cet instant, $D[m]$ correspond à la distance entre le mot trouvé et le mot pour lequel on cherche les plus proches. Il faut gérer la façon de garder les k mots les plus proches.

6.4.2 Optimisation clé : élagage (*pruning*)

À chaque nud, on regarde $\min(D)$. Si cette valeur est déjà trop élevée (supérieure au seuil souhaité), on **interrompt l'exploration**, car toute poursuite ne pourra qu'augmenter cette distance.

6.4.3 À implémenter

Implémenter une **méthode de la classe `Arbre`** prenant en paramètre un mot (sous forme de chaîne de caractères) et retournant les k mots les plus proches, dont la distance à ce mot est **inférieure à un seuil donné**.

6.5 Fonctionnalité 15 Correction d'un fichier texte

Écrire une **fonction** qui prend en argument un fichier texte. Pour chaque mot absent du dictionnaire, la fonction propose à l'utilisateur k mots suffisamment proches. L'utilisateur peut :

- choisir l'un des mots proposés ;
- valider le mot d'origine (celui-ci est alors ajouté au dictionnaire).

La nouvelle version du fichier est stockée dans un autre fichier ayant le même nom que le fichier d'origine avec l'extension `.cor`.

Remarque

Le texte du fichier peut être rangé dans une liste chaînée (par exemple).

6.6 Fonctionnalité 16 Interface interactive en terminal

Proposer une version interactive simple utilisant le terminal.

7 Récapitulatif des fonctionnalités

N°	Description	Statut
1	Constructeur sans argument	Obligatoire
2	Constructeur avec fichier	Obligatoire
3	Destructeur	Obligatoire
4	Constructeur par recopie	Obligatoire
5	Insertion d'un mot	Obligatoire
6	Recherche exacte	Obligatoire
7	Suppression d'un mot	Obligatoire
8	Comptage des mots	Obligatoire
9	Longueur du mot le plus long	Obligatoire
10	Sauvegarde dans fichier	Obligatoire
11	Autocomplétion par préfixe	Avancé
12	Saisie de texte assistée	Avancé
13	Distance de Levenshtein	Avancé
14	Correction via l'arbre	Avancé
15	Correction d'un fichier texte	Avancé
16	Interface interactive terminal	Avancé

TABLE 2 – Récapitulatif des 16 fonctionnalités